Django Advanced Notes: Database Transactions, Custom Managers & ORM Optimization

Prepared by: Faruk Ahmed

Topics: Django ORM, Database Transactions, Query Optimization

Level: Beginner to Advanced

Language: English

Part 1: Database Transactions

1. What Is a Database Transaction?

A **Database Transaction** is a logical unit of one or more database operations that must either succeed together or be rolled back when necessary if a failure occurs.

A transaction may include the following SQL operations:

- `INSERT`
- `UPDATE`
- `DELETE`
- `SELECT`

However, `SELECT` generally does not modify database data. Not all database operations behave identically within a transaction; their behavior depends on the database backend and transaction configuration.

Simple Example: Bank Transfer

Suppose we need to transfer 500 units of currency from Account A to Account B.

Two operations are required:

1. Deduct 500 from Account A.
2. Add 500 to Account B.

If the first operation succeeds but the second fails, Account A may lose the money without Account B receiving it.

Using a transaction allows both operations to be handled as part of the same transaction.

- If all operations succeed: `COMMIT`
- If the transaction fails: `ROLLBACK`

SQL Transaction Example

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 500
WHERE id = 1;

UPDATE accounts
SET balance = balance + 500
WHERE id = 2;

COMMIT;
```

Here, `COMMIT` makes the transaction's changes permanent.

If an error occurs before the transaction is committed:

```
ROLLBACK;
```

Remember: Using a transaction does not automatically handle every type of error. The application must implement appropriate error handling and ensure that rollback occurs when necessary.

---

2. ACID Properties

The four important properties of database transactions are known as ACID.

ACID stands for:

- Atomicity
- Consistency
- Isolation
- Durability

2.1 Atomicity

Atomicity means that all changes within a transaction are treated as a single logical unit.

If all operations succeed, the transaction is committed. If an operation fails, the transaction is rolled back so that partial changes from the transaction are not left behind.

Example:

```
Account A: 2000
Account B: 1000

Transfer: 500

After a successful transfer:
Account A = 1500
Account B = 1500

If the transfer fails:
Account A = 2000
Account B = 1000
```

2.2 Consistency

Consistency means that the database's defined constraints and business rules must remain valid before and after a transaction.

Examples include:

- Maintaining account balance rules.
- Enforcing foreign key constraints.
- Preventing duplicate values in unique fields.

Suppose a database rule specifies that an account balance must never be negative.

```
from django.db import models
from django.db.models import Q


class Account(models.Model):
    balance = models.DecimalField(
        max_digits=12,
        decimal_places=2,
    )

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=Q(balance__gte=0),
                name="account_balance_nonnegative",
            )
        ]
```

This constraint helps prevent negative balances at the database level.

Remember: Transactions alone cannot enforce every business rule. Appropriate database constraints and application logic are also necessary.

2.3 Isolation

Isolation controls unwanted interference between multiple transactions running concurrently.

The database determines how concurrent transactions interact according to the configured isolation level.

For example, two customers may attempt to purchase the last available item at the same time. Without appropriate concurrency control, the stock quantity could be updated incorrectly.

The actual isolation behavior depends on the database backend and the isolation level being used.

2.4 Durability

Durability means that once a transaction has been successfully committed, its changes remain stored according to the database's durability guarantees.

For example:

```
Payment successful
        ↓
Transaction committed
        ↓
Order status = PAID
```

After a successful commit, an ordinary application error does not automatically roll back the committed changes.

However, actual durability depends on database configuration, storage, and failure-recovery mechanisms.

---

3. Database Transactions in Django

Django primarily uses the following module to manage database transactions:

```
from django.db import transaction
```

3.1 `transaction.atomic()`

`transaction.atomic()` helps manage database operations within a single transaction.

```
from django.db import transaction


with transaction.atomic():
    # Database operations
    pass
```

When the block completes successfully, the outermost transaction is committed. If an exception escapes the block, the relevant transaction or savepoint is rolled back.

3.2 Bank Transfer Example

Suppose the `Account` model contains a field named `balance`.

```
from django.db import transaction
from django.db.models import F

from .models import Account


def transfer_money(from_id, to_id, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive.")

    with transaction.atomic():
        sender = Account.objects.get(pk=from_id)

        if sender.balance < amount:
            raise ValueError("Insufficient balance.")

        updated = Account.objects.filter(
            pk=from_id,
            balance__gte=amount,
        ).update(
            balance=F("balance") - amount
        )

        if updated != 1:
            raise ValueError("Unable to debit account.")
```

```
            recipient_updated = Account.objects.filter(
                pk=to_id,
            ).update(
                balance=F("balance") + amount
            )

            if recipient_updated != 1:
                raise Account.DoesNotExist(
                    "Recipient account not found."
                )
```

How does it work?

1. Validates that the amount is positive.
2. Checks the sender's balance.
3. Deducts money from the sender at the database level.
4. Adds money to the recipient's balance.
5. Commits the transaction if all operations succeed.
6. Rolls back the transaction if an exception escapes the transaction block.

Using `F()` expressions and conditional updates allows balance changes to be performed at the database level.

Production note: Real financial applications must also consider concurrent transfers, account locking, currency precision, idempotency, transaction isolation, and ledger design. The example above is an educational starting point, not a complete banking implementation.

3.3 Proper Exception Handling

In general, handling exceptions outside the transaction block is a good practice.

```
from django.db import transaction, DatabaseError


try:
    with transaction.atomic():
        # Database operations
        perform_database_updates()
```

```
    except DatabaseError:
        print("Database operation failed.")
```

Here, `perform_database_updates()` is a placeholder; replace it with your own function.

Important: After a database error occurs, you should not attempt another query inside the same broken transaction. Exit the `atomic()` block first so that the rollback can complete.

Avoid catching every exception indiscriminately because doing so may hide programming errors.

---

4. Nested Transactions and Savepoints

You can use one `atomic()` block inside another.

```
from django.db import transaction


with transaction.atomic():
    # Outer transaction

    with transaction.atomic():
        # Inner transaction / savepoint
        pass
```

Normally, a nested `atomic()` block creates a savepoint.

A savepoint allows a specific part of a transaction to be rolled back without necessarily rolling back the entire outer transaction.

Savepoint Example

```
from django.db import transaction


with transaction.atomic():
    create_order()

    try:
        with transaction.atomic():
            create_optional_coupon()
    except ValueError:
```

```
            print("Coupon operation failed.")

        create_order_history()
```

If `create_optional_coupon()` raises a `ValueError`, changes made inside the inner block are rolled back. Because the exception is caught outside the inner block, the outer transaction can continue.

However, if another uncaught error occurs in the outer transaction, the entire outer transaction will also be rolled back.

Remember: Rolling back to a savepoint and committing the outer transaction are two different things.

---

5. Django Auto-commit Mode

Django normally operates in autocommit mode.

This means that outside an explicit transaction, each successful database operation is generally committed immediately.

```
Account.objects.filter(pk=1).update(balance=1000)
```

If this operation is not part of an explicit transaction, it is normally committed after it succeeds.

To commit or roll back multiple related operations together:

```
with transaction.atomic():
    operation_one()
    operation_two()
    operation_three()
```

The functions above are illustrative placeholders.

`ATOMIC_REQUESTS`

Setting `ATOMIC_REQUESTS = True` for a database in Django settings allows Django to wrap each view's execution in a transaction.

However, this does not cover every part of request processing or every database operation in the same way. Middleware and template rendering may execute outside the transaction.

Wrapping every view in a transaction introduces overhead, so choose this configuration according to your application's requirements.

Part 2: Django Custom Model Manager and QuerySet

1. What Is a Model Manager?

A Django Model Manager is the primary interface for initiating database queries for a model.

Django models normally have a default manager named `objects`.

```
Product.objects.all()
Product.objects.filter(is_active=True)
Product.objects.get(pk=1)
```

A manager allows you to retrieve model instances or QuerySets from the database.

2. What Is a Custom Model Manager?

A **Custom Model Manager** is a manager created by subclassing `models.Manager`, allowing you to add your own query logic and reusable methods.

Custom managers help you:

- Keep repeated query logic in one place.
- Make code cleaner and more reusable.
- Organize model-related query methods.
- Retrieve active, published, or featured objects easily.

---

3. Creating a Custom Manager

Model Example

```
from django.db import models


class ActiveProductManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(
            is_active=True
        )
```

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
    is_active = models.BooleanField(default=True)

    objects = models.Manager()
    active = ActiveProductManager()

    def __str__(self):
        return self.name
```

Usage

```
all_products = Product.objects.all()

active_products = Product.active.all()
```

`Product.objects.all()` returns all products.

`Product.active.all()` returns only products where `is_active=True`.

Example Output

Suppose the database contains:

id	name	is_active
1	Laptop	True
2	Mouse	False
3	Keyboard	True

Then:

```
Product.objects.all()
```

Result: Laptop, Mouse, Keyboard

On the other hand:

```
Product.active.all()
```

Result: Laptop, Keyboard

Warning: Filtering the default manager can cause objects to be unexpectedly excluded from certain operations. Choosing a default manager can also affect Django migrations, related model access, and other framework behavior.

---

4. Custom Methods Inside a Custom Manager

You can add your own methods to a manager without overriding `get_queryset()`.

```
from django.db import models


class ProductManager(models.Manager):
    def active_products(self):
        return self.get_queryset().filter(
            is_active=True
        )

    def inactive_products(self):
        return self.get_queryset().filter(
            is_active=False
        )

    def expensive_products(self, minimum_price=1000):
        return self.get_queryset().filter(
            price__gt=minimum_price
        )


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
    is_active = models.BooleanField(default=True)

    objects = ProductManager()
```

Usage

```
active = Product.objects.active_products()

inactive = Product.objects.inactive_products()

expensive = Product.objects.expensive_products(5000)
```

Here, `expensive_products(5000)` returns products priced above 5000.

Manager Methods vs. QuerySet Methods

Manager methods are generally used to start queries through the manager.

```
Product.objects.active_products()
```

However, if you want custom methods to support chaining on QuerySets, using a QuerySet subclass is often a better approach.

---

5. Custom QuerySet

Subclassing `QuerySet` allows you to create reusable, chainable filtering methods.

```
from django.db import models


class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)

    def inactive(self):
        return self.filter(is_active=False)

    def expensive(self, minimum_price=1000):
        return self.filter(price__gt=minimum_price)


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

```
    is_active = models.BooleanField(default=True)

    objects = ProductQuerySet.as_manager()
```

Usage

```
Product.objects.active()

Product.objects.inactive()

Product.objects.active().expensive(5000)
```

The last query returns active products priced above 5000.

Why Use `as_manager()`?

```
objects = ProductQuerySet.as_manager()
```

This exposes suitable public QuerySet methods through the Manager interface.

It avoids writing the same query logic separately in both the Manager and QuerySet.

---

6. Difference Between Manager and QuerySet

Feature	Manager	QuerySet
Main purpose	Start queries	Build and filter queries
Typical access	`Product.objects`	`Product.objects.all()`
Custom methods	Supported	Supported
Chaining	Depends on the method's	Chainable when methods

	return value	return QuerySets
Reusable filtering	Supported	Often more convenient for reusable filtering logic
Created by	Subclassing `models.Manager`	Subclassing `models.QuerySet`

Golden Rule

- Use a Manager to provide a query entry point.
- Use a QuerySet for reusable and chainable filtering logic.
- Avoid duplicating the same logic in both places.

---

7. Best Practices for Custom Managers

7.1 Separate Business Logic and Query Logic

Query-related behavior can be placed in Managers or QuerySets.

However, complex business workflows do not have to live inside a Manager. Depending on the application, a service layer may be more appropriate.

7.2 Choose the Default Manager Carefully

Example:

```
class Product(models.Model):
    objects = models.Manager()
    active = ActiveProductManager()
```

Here, `objects` is the default manager.

To retrieve all objects:

```
Product.objects.all()
```

To retrieve only active objects:

```
Product.active.all()
```

This arrangement reduces the risk of unexpectedly filtering normal default queries.

7.3 Return QuerySets from QuerySet Methods

```
def active(self):
    return self.filter(is_active=True)
```

Returning a QuerySet allows chaining:

```
Product.objects.active().order_by("name")
```

Part 3: Django ORM Optimization — Basic to Pro Level

1. What Is ORM Optimization?

Django ORM Optimization is the process of improving application performance by reducing database query counts, query execution costs, transferred data, and unnecessary Python-side processing.

Suppose an e-commerce page needs to display 100 products.

If the application separately queries each product's category, images, and reviews, it may generate a large number of database queries.

The goal of optimization is not simply to minimize query counts. Total execution time, memory consumption, and database workload must also be considered.

---

2. The N+1 Query Problem

The N+1 Query Problem occurs when an application first retrieves multiple objects using one query and then executes an additional query for related data for each object.

Example Models

```
from django.db import models
```

```
class Author(models.Model):
    name = models.CharField(max_length=100)


class Book(models.Model):
    title = models.CharField(max_length=200)

    author = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name="books",
    )
```

Without Optimization

```
books = Book.objects.all()

for book in books:
    print(book.title, book.author.name)
```

Suppose there are 100 books and each book's author relationship is accessed.

In a typical case:

```
1 query   → Books
100 queries → Authors

Total: 101 queries
```

If multiple books share the same author, relationship caching may reduce the actual query count. Nevertheless, the N+1 problem can still occur.

Why Is It a Problem?

- Database round trips increase.
- Page responses may become slower.
- API latency increases.
- Database connections and server resources are used more heavily.

The main solutions are `select_related()` and `prefetch_related()`.

---

3. `select_related()`

`select_related()` is primarily used to retrieve related objects through the same SQL query for single-valued relationships.

Typical relationships include:

- `ForeignKey`
- `OneToOneField`

It uses SQL `JOIN` operations.

Example

```
books = Book.objects.select_related("author")

for book in books:
    print(book.title, book.author.name)
```

Django can now retrieve the required book and author data in the same query.

Conceptual SQL

```
SELECT
    book.id,
    book.title,
    book.author_id,
    author.id,
    author.name
FROM book
INNER JOIN author
    ON book.author_id = author.id;
```

This is conceptual SQL. The actual SQL generated by Django may vary depending on the database backend and model configuration.

Multiple Relationships

Suppose we have:

```
class Order(models.Model):
    user = models.ForeignKey(
        "auth.User",
        on_delete=models.CASCADE,
    )
```

```
class Payment(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
    )
```

Query:

```
payments = Payment.objects.select_related(
    "order__user"
)

for payment in payments:
    print(payment.order.user.username)
```

The `Payment → Order → User` relationship can be loaded in the same SQL query.

Multiple ForeignKeys

```
orders = Order.objects.select_related(
    "user",
    "shipping_address",
)
```

This is useful only if `shipping_address` is an appropriate single-valued relationship field on the model.

When Should You Use It?

- When you need an order's user.
- When you need a book's author.
- When you need a product's category.
- When you need a payment's order and user.

Warning: Adding unnecessary `select_related()` calls to long relationship chains can produce complex joins and transfer more data than necessary.

---

4. `prefetch_related()`

`prefetch_related()` is used primarily for relationships that can contain multiple related objects.

Common use cases include:

- `ManyToManyField`

- Reverse `ForeignKey`
- Appropriate reverse `OneToOneField` access patterns

It generally executes multiple queries and combines the related objects with their parent objects in Python.

Example: Product Images

```
class Product(models.Model):
    name = models.CharField(max_length=100)


class ProductImage(models.Model):
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE,
        related_name="images",
    )

    image = models.ImageField(upload_to="products/")
```

Without Optimization

```
products = Product.objects.all()

for product in products:
    for image in product.images.all():
        print(image.image.url)
```

Suppose there are 100 products.

A typical query pattern is:

```
1 query   → Products
100 queries → Images

Total: 101 queries
```

With Optimization

```
products = Product.objects.prefetch_related("images")

for product in products:
```

```
        for image in product.images.all():
            print(image.image.url)
```

A typical query pattern becomes:

```
1 query → Products
1 query → Images

Total: 2 queries
```

This is a simplified example. Query counts may differ depending on dataset size, batching, nested relationships, and backend behavior.

ManyToMany Example

```
class Book(models.Model):
    title = models.CharField(max_length=200)

    authors = models.ManyToManyField(
        Author,
        related_name="books",
    )
```

Query:

```
books = Book.objects.prefetch_related("authors")

for book in books:
    print(book.title)

    for author in book.authors.all():
        print(author.name)
```

Django generally executes separate queries for books and authors, then associates the results in Python.

Multiple Prefetch Operations

```
products = Product.objects.prefetch_related(
    "images",
    "reviews",
)
```

If `reviews` is a valid related name, both images and reviews can be prefetched.

---

5. `Prefetch()` — Advanced Custom Prefetching

`Prefetch()` provides more control over prefetch operations.

It is useful when you need to:

- Filter related objects.
- Order related objects explicitly.
- Store related data in a separate attribute.
- Apply `select_related()` or other optimizations to the related queryset.

Example Model

```
class Review(models.Model):
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE,
        related_name="reviews",
    )

    rating = models.PositiveSmallIntegerField()
```

5.1 Filtered Prefetch

To retrieve only reviews with a rating of 4 or higher:

```
from django.db.models import Prefetch


products = Product.objects.prefetch_related(
    Prefetch(
        "reviews",
        queryset=Review.objects.filter(
            rating__gte=4
        ),
    )
)
```

The prefetched related-object cache will contain only reviews rated 4 or higher.

Important: After this filtered prefetch, `product.reviews.all()` can use the prefetched cache. However, adding a different filter may require a new query.

5.2 Using `to_attr`

Using `to_attr` is often preferable when you want to store filtered related data in a separate attribute.

```
products = Product.objects.prefetch_related(
    Prefetch(
        "reviews",
        queryset=Review.objects.filter(
            rating__gte=4
        ),
        to_attr="good_reviews",
    )
)

for product in products:
    print(product.good_reviews)
```

Here, `good_reviews` is a Python list.

Benefits include:

- Filtered data is clearly separated.
- The original related manager retains its normal semantics.
- Iteration over the list is straightforward.
- Confusion is reduced when both all reviews and filtered reviews are needed.

5.3 Nested Prefetch

Suppose `OrderItem` has `order` and `product` relationships.

```
class OrderItem(models.Model):
    order = models.ForeignKey(
        "Order",
        on_delete=models.CASCADE,
        related_name="items",
    )

    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE,
    )
```

Query:

```
orders = Order.objects.prefetch_related(
    "items__product"
)
```

This prefetches orders, their items, and the products associated with those items.

A potentially more efficient combination is:

```
orders = Order.objects.prefetch_related(
    Prefetch(
        "items",
        queryset=OrderItem.objects.select_related(
            "product"
        ),
    )
)
```

In this example:

- One query may retrieve the orders.
- Another query may retrieve the order items joined with their products.

This combination can provide a more efficient query plan for certain nested relationships.

---

6. `select_related()` vs. `prefetch_related()`

Feature	`select_related()`	`prefetch_related()`
Primary use	ForeignKey, OneToOne	ManyToMany, Reverse FK, other suitable relationships
SQL JOIN	Uses SQL JOIN	Generally uses

		separate queries for related collections
Query count	Usually one main query	Usually multiple queries
Data combination	Performed by the database	Relationships are mapped in Django/Python
Best use case	Single related object	Multiple related objects
Custom filtered related data	Not its primary purpose	Supports filtering through `Prefetch()`

Simple Rule

```
ForeignKey / OneToOne
        ↓
select_related()

ManyToMany / Reverse ForeignKey
        ↓
prefetch_related()

Filtered related objects
```

```
          ↓
Prefetch()
```

Part 4: Django ORM Optimization — 10 Advanced Techniques

1. `annotate()` and Aggregation

`annotate()` adds calculated values to individual objects in a queryset.

Example: Product Review Count

```
from django.db.models import Count



products = Product.objects.annotate(
    review_count=Count("reviews")
)

for product in products:
    print(product.name, product.review_count)
```

Here, `reviews` is the `related_name` on the `Product` model.

Django can perform the aggregation at the database level, avoiding the need to execute a separate count query for every product.

Conceptual SQL

```
SELECT
    product.id,
    product.name,
    COUNT(review.id) AS review_count
FROM product
LEFT JOIN review
    ON review.product_id = product.id
GROUP BY product.id, product.name;
```

This is illustrative SQL. The actual query may vary depending on the backend and model configuration.

`annotate()` vs. `aggregate()`

`annotate()` adds a calculated value to each returned object.

```
Product.objects.annotate(
    review_count=Count("reviews")
)
```

`aggregate()` calculates summary values across the entire queryset.

```
from django.db.models import Avg, Count


summary = Product.objects.aggregate(
    average_price=Avg("price"),
    total_products=Count("id"),
)
```

Example result:

```
{
    "average_price": Decimal("2500.00"),
    "total_products": 20,
}
```

The actual result depends on the database contents.

Important Warning

Joining multiple one-to-many relationships in the same query can duplicate rows and produce incorrect counts or sums.

When appropriate, you may use:

```
Count("reviews", distinct=True)
```

However, `distinct=True` is not a universal solution to every aggregation problem. Always inspect the generated SQL and verify the expected results.

---

2. `only()` — Load Required Fields

`only()` allows you to load specific fields initially.

```
products = Product.objects.only(
    "id",
    "name",
)
```

Accessing another deferred field later may cause Django to execute an additional query.

Example

```
product = Product.objects.only(
    "id",
    "name",
).get(pk=1)

print(product.name)
```

The `name` field has already been loaded.

However:

```
print(product.description)
```

If `description` is deferred, Django may execute an additional query to retrieve it.

Best practice: Use `only()` when you need just a few fields and do not expect to access the deferred fields later.

---

3. `defer()` — Exclude Unnecessary Fields

`defer()` allows you to exclude selected fields from the initial query.

```
products = Product.objects.defer("description")
```

This can be useful when you do not need to load large text fields or other unnecessary data.

However, accessing a deferred field later may trigger an additional query.

Warning: Using `only()` or `defer()` does not automatically improve performance. Evaluate the query plan, field sizes, and actual access patterns before applying them.

---

4. `values()` and `values_list()`

When you need field values instead of model instances, use `values()` or `values_list()`.

`values()`

```
products = Product.objects.values(
    "id",
    "name",
```

```
    "price",
)
```

Example output:

```
[
    {"id": 1, "name": "Laptop", "price": Decimal("50000.00")},
    {"id": 2, "name": "Phone", "price": Decimal("20000.00")},
]
```

`values_list()`

```
names = Product.objects.values_list(
    "name",
    flat=True,
)
```

Example output:

```
["Laptop", "Phone", "Keyboard"]
```

Differences

Method	Output
`values()`	Dictionary-like rows
`values_list()`	Tuple-based rows
`values_list(flat=True)`	A sequence of single-field values

If you only need identifiers:

```
product_ids = Product.objects.values_list(
    "id",
    flat=True,
)
```

This can avoid the overhead of constructing model instances when only the specified field values are required.

---

5. `F()` Expressions

`F()` expressions allow calculations using the current value of a database field without loading that value into Python first.

Example: Decrease Stock

```
from django.db.models import F


Product.objects.filter(
    pk=1,
).update(
    stock=F("stock") - 1
)
```

The stock value is decreased at the database level.

Benefits

- Avoids loading the field value into Python.
- Reduces the risk of read-modify-write race conditions.
- Performs arithmetic at the database level.
- Is useful for bulk update operations.

Conditional Stock Update

```
updated = Product.objects.filter(
    pk=1,
    stock__gt=0,
).update(
    stock=F("stock") - 1
)
```

```
if updated == 0:
    print("Product not found or stock unavailable.")
```

The `stock > 0` condition is included in the same SQL update operation.

Warning: `F()` expressions do not solve every concurrency problem. Complex workflows may require transactions, database constraints, or locking.

---

6. `Subquery()` and `OuterRef()`

`Subquery()` allows the result of one query to be used as an expression in another query.

`OuterRef()` references a field from the current row of the outer query.

Example: Latest Review Rating

```
from django.db.models import OuterRef, Subquery


latest_review = Review.objects.filter(
    product=OuterRef("pk")
).order_by("-id")

products = Product.objects.annotate(
    latest_rating=Subquery(
        latest_review.values("rating")[:1]
    )
)
```

Now:

```
for product in products:
    print(product.name, product.latest_rating)
```

Each product receives the rating from its review with the highest `id`.

Important: In real projects, using `created_at` and, when necessary, a unique tie-breaker is generally more appropriate for identifying the latest review.

For example:

```
latest_review = Review.objects.filter(
    product=OuterRef("pk")
).order_by("-created_at", "-pk")
```

You can then use the same `Subquery()` pattern.

If a product has no reviews, `latest_rating` will generally be `None`.

---

7. `Exists()` — Check Whether Related Records Exist

`Exists()` checks whether a related record exists.

Example: Check Whether a Product Has Reviews

```
from django.db.models import Exists, OuterRef


reviews = Review.objects.filter(
    product=OuterRef("pk")
)

products = Product.objects.annotate(
    has_review=Exists(reviews)
)
```

Usage:

```
for product in products:
    print(product.name, product.has_review)
```

Example output:

```
Laptop True
Phone True
Keyboard False
```

This is illustrative output.

Retrieve Only Products with Reviews

```
products_with_reviews = Product.objects.filter(
    Exists(reviews)
)
```

In this case, you do not need to create a boolean annotation for every product; you can filter directly.

`Exists()` allows the database to use an appropriate query plan for checking whether matching records exist.

---

8. `bulk_create()` — Create Multiple Objects at Once

`bulk_create()` is efficient for inserting many objects.

Inefficient Pattern

```
for i in range(1000):
    Product.objects.create(
        name=f"Product {i}",
        price=100,
    )
```

This generally executes a separate insert query for each object.

Optimized Pattern

```
products = [
    Product(
        name=f"Product {i}",
        price=100,
    )
    for i in range(1000)
]

Product.objects.bulk_create(
    products,
    batch_size=500,
)
```

With `batch_size=500`, Django can insert the objects in multiple batches as needed.

Therefore, it is not guaranteed to execute everything in a single SQL query.

Important Limitations

Generally, `bulk_create()` does not call each object's `save()` method and does not trigger normal per-instance save signals in the same way.

If your business logic depends on a model's `save()` method or signals, verify that logic before using bulk operations.

---

9. `bulk_update()` — Update Multiple Objects at Once

`bulk_update()` can update selected fields on multiple objects.

```
products = list(Product.objects.filter(
    is_active=True
))

for product in products:
    product.price += 10

Product.objects.bulk_update(
    products,
    ["price"],
    batch_size=500,
)
```

Here, the prices of active products are modified in Python and then updated in bulk.

Important Considerations

- Loading the objects may require an initial query.
- `bulk_update()` is not guaranteed to finish in a single SQL query.
- It does not call each instance's `save()` method.
- Do not rely on normal model save signals being triggered.
- Consider memory consumption when processing large datasets.

If you only need to increase a database field's value, an `update()` operation using an `F()` expression may be simpler.

```
Product.objects.filter(
    is_active=True
).update(
    price=F("price") + 10
)
```

---

10. Database Indexing

Database indexes can improve query performance, particularly for suitable fields used in `WHERE`, `JOIN`, and `ORDER BY` operations.

Single-field Index

```
class Product(models.Model):
    name = models.CharField(
        max_length=100,
```

```
            db_index=True,
        )
```

Using `Meta.indexes`

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    class Meta:
        indexes = [
            models.Index(
                fields=["name", "price"],
                name="product_name_price_idx",
            ),
        ]
```

This creates a composite index.

When Are Indexes Useful?

- Fields frequently used for filtering.
- Searching for specific records in large tables.
- Appropriate foreign key and join-related queries.
- Sorting patterns identified in real workloads.

Index Trade-offs

- Indexes require additional storage.
- Insert and update operations incur index-maintenance overhead.
- Adding an index to every field does not necessarily improve performance.
- The order of fields in a composite index matters.

Best practice: Create indexes based on actual workloads and query execution plans in your production database.

Part 5: Additional Production-Level Optimization

1. `select_for_update()` — Row Locking

On supported database backends, `select_for_update()` helps lock selected rows during a transaction.

It can be useful when handling concurrent updates.

Example: Stock Reservation

```
from django.db import transaction


def reserve_stock(product_id, quantity):
    if quantity <= 0:
        raise ValueError("Quantity must be positive.")

    with transaction.atomic():
        product = (
            Product.objects.select_for_update()
            .get(pk=product_id)
        )

        if product.stock < quantity:
            raise ValueError("Insufficient stock.")

        product.stock -= quantity
        product.save(update_fields=["stock"])
```

This attempts to lock the product row inside the transaction.

Why Is `atomic()` Necessary?

`select_for_update()` generally needs to be used inside a transaction. The exact locking behavior varies depending on the database backend.

Important Considerations

- SQLite does not provide the same row-level locking behavior as databases such as PostgreSQL.
- Avoid holding locks while performing lengthy operations.
- Consider deadlocks and lock contention.
- A conditional update using an `F()` expression may be a simpler alternative in some cases.

---

2. QuerySet Lazy Evaluation

Django QuerySets are generally lazy.

This means that creating a QuerySet does not immediately execute a database query.

```
products = Product.objects.filter(
    is_active=True
)
```

At this point, the query generally has not been executed.

However:

```
for product in products:
    print(product.name)
```

The QuerySet is evaluated when iteration begins.

Other evaluation triggers include:

```
list(products)
len(products)
bool(products)
```

Methods such as `count()` and `exists()` generally execute appropriate database queries.

Example: `exists()` vs. `count()`

To check whether any records exist:

```
if Product.objects.filter(is_active=True).exists():
    print("Active products found.")
```

To retrieve the number of records:

```
count = Product.objects.filter(
    is_active=True
).count()
```

When you only need to check existence, `exists()` is more appropriate than `count() > 0`.

---

3. Pagination

Pagination allows you to retrieve large datasets in smaller portions instead of loading everything at once.

Example

```
from django.core.paginator import Paginator


products = Product.objects.order_by("id")

paginator = Paginator(products, 20)

page = paginator.get_page(1)

for product in page:
    print(product.name)
```

This displays 20 products per page.

Important Considerations

Offset-based pagination can become inefficient for very large page numbers. For large datasets, consider appropriate indexing and cursor-based pagination.

---

4. `iterator()` and Memory Usage

When iterating over large querysets, `iterator()` can help control memory usage by avoiding the normal QuerySet result cache.

```
for product in Product.objects.iterator(
    chunk_size=1000
):
    process_product(product)
```

Here, `process_product()` is a placeholder function.

`iterator()` generally bypasses the normal QuerySet result cache. However, memory usage and query behavior should be evaluated according to the database driver, backend, and related-data access patterns.

---

5. Inspecting Queries with `connection.queries`

During development, you can inspect executed queries using:

```
from django.db import connection
```

```
print(connection.queries)
```

When `DEBUG=True`, this can display information about executed queries on the current connection.

Important Considerations

- It is not a replacement for production query monitoring.
- Query logs may contain sensitive data.
- Behavior can vary depending on the connection and backend.

Django Debug Toolbar

Django Debug Toolbar allows you to inspect SQL queries, query timings, and duplicate-query information in the development browser.

It is particularly useful for identifying N+1 query problems.

---

6. `QuerySet.explain()`

Inspecting the generated SQL execution plan is important when optimizing queries.

```
queryset = Product.objects.filter(
    is_active=True
)

print(queryset.explain())
```

Depending on backend support, the execution plan may show scans, index usage, joins, and other planning details.

The output of `EXPLAIN` varies by database backend.

In general, compare execution plans before and after applying optimizations.

Part 6: Real E-commerce Optimization Example

Suppose an e-commerce application contains the following models:

- `Product`
- `Category`
- `ProductImage`
- `Review`

Each product has a `ForeignKey` relationship with `Category` and reverse foreign-key relationships with images and reviews.

Optimized Query

```
from django.db.models import Count, Prefetch



products = (
    Product.objects
    .select_related("category")
    .prefetch_related(
        "images",
        Prefetch(
            "reviews",
            queryset=Review.objects.filter(
                rating__gte=4
            ),
            to_attr="good_reviews",
        ),
    )
    .annotate(
        review_count=Count(
            "reviews",
            distinct=True,
        )
    )
)
```

What Has Been Optimized?

1. `select_related("category")` — Attempts to retrieve category data in the same query.
2. `prefetch_related("images")` — Prefetches product images.
3. `Prefetch("reviews", ...)` — Stores only reviews rated 4 or higher in a separate `good_reviews` attribute.
4. `annotate()` — Adds the review count to each product.

Template Example

```
{% for product in products %}
    <h2>{{ product.name }}</h2>
```

```
        <p>Category: {{ product.category.name }}</p>

        <p>Review Count: {{ product.review_count }}</p>

        {% for image in product.images.all %}
            <img
                src="{{ image.image.url }}"
                alt="{{ product.name }}"
            >
        {% endfor %}

        <h3>Good Reviews</h3>

        {% for review in product.good_reviews %}
            <p>Rating: {{ review.rating }}</p>
        {% endfor %}
    {% endfor %}
```

Here, `images` is the related name for product images, and `reviews` is the related name for product reviews.

Important Considerations

Using `distinct=True` helps reduce the risk of counting the same review multiple times because of other joins.

However, complex aggregations must still be verified by inspecting the query plan and checking the results. Although prefetching can reduce query counts, loading every related object into memory at once may increase memory consumption.

Part 7: Django ORM Golden Rules

Requirement	Recommended Approach
Related ForeignKey object	`select_related()`

OneToOne relationship	`select_related()`
ManyToMany relationship	`prefetch_related()`
Reverse ForeignKey relationship	`prefetch_related()`
Filtered related objects	`Prefetch()`
Related object count	`annotate(Count(...))`
Aggregate summary	`aggregate()`
Database-level arithmetic	`F()`
Nested correlated query	`Subquery()` + `OuterRef()`

Existence check	`Exists()` or `.exists()`
Large insert	`bulk_create()`
Large object update	`bulk_update()`
Direct field update	`QuerySet.update()`
Exclude unnecessary fields	`only()` / `defer()`
Dictionary-style results	`values()`
Flat sequence of values	`values_list(flat=True)`
Appropriate query indexing	Database indexes + query-plan analysis
Concurrent row	Transactions +

updates	appropriate locking or conditional updates
Query debugging	Django Debug Toolbar / `QuerySet.explain()`

Part 8: Common Mistakes

Mistake 1: Using `select_related()` for Every Relationship

`select_related()` is primarily designed for single-valued relationships.

For ManyToMany relationships and reverse ForeignKey collections, `prefetch_related()` is generally the appropriate choice.

Mistake 2: Applying Another Filter After `prefetch_related()`

```
products = Product.objects.prefetch_related("reviews")
```

Then:

```
product.reviews.filter(rating__gte=4)
```

This new filter generally requires another database query.

If you only need filtered reviews, use `Prefetch()` to filter them in advance.

Mistake 3: Assuming `bulk_create()` Calls `save()` and Triggers Signals

Bulk operations do not follow the normal per-instance save lifecycle.

Use them carefully when business logic or signals depend on that lifecycle.

Mistake 4: Creating Indexes on Every Field

Indexes can improve query performance, but they also increase storage requirements and write overhead.

Choose indexes according to actual query workloads.

Mistake 5: Performing Critical Updates Without `transaction.atomic()`

Transactions may be necessary to maintain consistency across multiple dependent operations.

Mistake 6: Catching Every Exception

```
try:
    ...
except:
    pass
```

Ignoring errors this way can create data-integrity problems and make debugging difficult.

Handle specific exceptions appropriately.

Mistake 7: Assuming Query Count Is the Only Performance Metric

Even a query with a low query count can be extremely expensive.

Consider query count, execution time, execution plans, transferred data, and memory consumption together.

Part 9: Interview Questions

Q1. What is a database transaction?

A database transaction is a logical unit of one or more database operations that are committed together or rolled back when necessary if a failure occurs.

Q2. What are ACID properties?

- Atomicity
- Consistency
- Isolation
- Durability

Q3. What is `transaction.atomic()`?

It is a Django context manager or decorator used to manage database operations within a transaction.

Q4. What is a Django Model Manager?

A Model Manager is the interface used to initiate and organize database queries for a model.

Q5. What is the difference between Manager and QuerySet?

A Manager starts queries, while a QuerySet represents a collection of database results and provides filtering operations. QuerySet methods can generally be chained.

Q6. What is the N+1 query problem?

It occurs when one query retrieves parent objects and separate additional queries retrieve related data for individual objects.

Q7. What is the difference between `select_related()` and `prefetch_related()`?

`select_related()` uses SQL JOINs to retrieve single-valued relationships. `prefetch_related()` generally executes separate queries and combines related collections in Python.

Q8. What is `Prefetch()`?

It is a Django ORM utility that provides control over related querysets, filtering, and behavior such as `to_attr`.

Q9. What is the difference between `annotate()` and `aggregate()`?

`annotate()` adds calculated values to individual objects, while `aggregate()` returns summary values for an entire queryset.

Q10. What is an `F()` expression?

An `F()` expression performs calculations using the current value of a database field at the database level.

Q11. What is `Exists()`?

`Exists()` is an ORM expression that checks whether a subquery contains at least one matching row.

Q12. What is `bulk_create()`?

It is a Django API for inserting multiple model objects in bulk.

Q13. What is `bulk_update()`?

It is a Django API for updating selected fields on multiple model objects in bulk.

Q14. What is a database index?

A database index is a data structure that helps the database locate specific data more efficiently, at the cost of additional storage and write overhead.

Q15. How do you debug slow Django queries?

- Use Django Debug Toolbar.
- Inspect execution plans with `QuerySet.explain()`.
- Measure query counts and execution times.
- Apply `select_related()` and `prefetch_related()` where appropriate.
- Create indexes according to actual workloads.

Final Revision Checklist

- Understand database transactions and ACID properties.
- Use `transaction.atomic()` correctly.
- Understand the difference between rollback and nested savepoints.
- Create custom Model Managers.
- Understand custom QuerySets and `as_manager()`.
- Identify N+1 query problems.
- Choose between `select_related()` and `prefetch_related()`.
- Use `Prefetch()` for filtered related objects.
- Understand the difference between `annotate()` and `aggregate()`.
- Use `F()`, `Subquery()`, `OuterRef()`, and `Exists()`.
- Understand the limitations of `bulk_create()` and `bulk_update()`.
- Use `only()`, `defer()`, `values()`, and `values_list()`.
- Understand database indexes and query execution plans.
- Maintain data integrity when working with transactions and concurrent updates.

Final Advice: The most important habit in Django ORM optimization is to avoid optimizing based on assumptions. Inspect the queries first, identify the bottleneck, apply the necessary optimization, and measure performance again.